

# Software Requirements Specification (SRS)

## *Development of a Modern OpenGL Forward Renderer*

### 1. Introduction

#### 1.1 Purpose of This Document

This Software Requirements Specification defines the scope, architecture direction, functional requirements, non-functional requirements, delivery plan, and acceptance criteria for a one-term project whose purpose is to implement the graphics subsystem of a game engine as a modern OpenGL forward renderer.

The document is intended to be operational rather than aspirational. It is written so that a technical lead, supervisor, or student team can use it to guide execution, assess progress weekly, and evaluate whether the final system satisfies the agreed scope.

#### 1.2 Project Context

The renderer is the only engine subsystem in scope. It begins with basic window/context bootstrap and evolves into a structured forward-rendering architecture that supports imported assets, materials, direct lighting, directional shadow mapping, direct-light PBR, HDR post-processing, and pragmatic optimization features.

### 2. Overall Description

#### 2.1 Product Perspective

The target system is a reusable rendering subsystem.

#### 2.2 Product Goals

- Deliver a clean, modular C++ renderer using OpenGL on Windows.
- Implement a credible modern forward-rendering feature set within ten weekly sprints.
- Demonstrate direct-light PBR, directional shadows, HDR, tone mapping, bloom, culling, instancing, and diagnostics.
- Keep architecture extensible enough for future engine growth without committing to deferred rendering complexity in this version.
- Provide documentation, test evidence, benchmark results, and demo scenes suitable for academic evaluation.

#### 2.3 Constraints

- Programming language: C++.
- Graphics API/backend: OpenGL.
- Primary target platform: Windows desktop.

#### 2.4 Assumptions and Dependencies

- Team members have baseline familiarity with C++, Git, and shader development or can be onboarded quickly.
- At least one development GPU supports the OpenGL features required for shadow mapping, HDR render targets, and instancing.
- Third-party libraries are acceptable and may be statically or dynamically linked as appropriate.
- Reference assets and demo materials can be sourced or created without blocking the schedule.

## 3. Project Scope

### 3.1 In Scope

- Window and OpenGL context initialization
- Renderer lifecycle and configuration
- Shader system and buffer abstractions
- Built-in primitives and imported static meshes
- Texture loading and GPU resource caches
- Material system with direct-light metallic-roughness PBR and normal mapping
- Camera, transforms, scene submission API, and frame data flow
- Forward opaque rendering, with pragmatic transparency hooks
- Directional and point lights
- Directional shadow mapping; optional basic cascaded shadows as stretch
- Skybox/environment background
- HDR rendering, tone mapping, and bloom
- Frustum culling, instanced rendering, draw/state organization
- Debug UI, resource inspectors, render statistics, and benchmark presets
- Final documentation, demo scenes, and release packaging

### 3.2 Out of Scope

- Deferred rendering and any G-buffer pipeline
- SSAO, SSR, volumetrics, advanced GI, or full IBL-based PBR
- Skeletal animation, terrain, particles, and compute-driven effects
- Complete editor, gameplay systems, physics, audio, networking, or scripting
- Multi-backend abstraction to Vulkan/DirectX within this project window

### 3.3 Final Renderer Vision

By the end of the project, the final system should present as a technically credible modern OpenGL forward renderer: imported assets, textured and normal-mapped materials, directional and point lights, stable directional shadows, direct-light PBR shading, skybox, HDR output, tone mapping, bloom, culling, instancing, debug controls, resource diagnostics, and a final demo set.

## 4. Functional Requirements

Priority codes: C = Critical, H = High, M = Medium, S = Stretch.

| ID     | Title                         | Description                                                                                     | P | Acceptance Summary                                                         |
|--------|-------------------------------|-------------------------------------------------------------------------------------------------|---|----------------------------------------------------------------------------|
| FR-001 | Window Creation               | The system shall create and manage a resizable Windows desktop application window through GLFW. | C | Application opens, resizes, swaps buffers, and closes cleanly.             |
| FR-002 | OpenGL Context Initialization | The renderer shall initialize a valid OpenGL context with debug support in debug builds.        | C | Startup log reports vendor, renderer, version, and debug status.           |
| FR-003 | Renderer Lifecycle            | The system shall expose explicit initialize, resize, render, and shutdown operations.           | C | Renderer can start, render frames, resize resources, and shut down safely. |
| FR-004 | Shader Management             | The system shall load, compile, link, cache, and report errors for shader programs.             | C | Compile and link failures are readable and actionable.                     |
| FR-005 | Buffer and VAO Management     | The system shall manage vertex/index buffers and vertex array objects for reusable geometry.    | C | Built-in and imported meshes render through managed buffers.               |
| FR-006 | Primitive Geometry            | The renderer shall provide built-in primitives for testing and fallback content.                | H | Triangle, quad, cube, and at least one rounded primitive are available.    |
| FR-007 | Camera System                 | The system shall support perspective camera data and runtime movement controls.                 | C | Navigable 3D scene renders correctly under camera motion.                  |
| FR-008 | Transform Support             | The renderer shall accept per-object transforms and apply model matrices correctly.             | C | Objects render at correct position, orientation, and scale.                |
| FR-009 | Scene Submission API          | The renderer shall expose a render-item submission API compatible with future ECS integration.  | C | Sample application submits a frame without direct low-level GL draw code.  |
| FR-010 | Mesh Import                   | The system shall load imported static meshes through Assimp.                                    | H | At least one non-trivial external model renders correctly.                 |
| FR-011 | Texture Loading               | The system shall load 2D textures from disk and upload them to GPU.                             | C | Textures appear correctly on imported and primitive objects.               |
| FR-012 | Resource Caches               | The system shall cache reusable textures, meshes, and shader programs.                          | H | Duplicate asset requests reuse existing managed resources.                 |
| FR-013 | Material System               | The renderer shall support data-driven materials with shader selection and parameters.          | C | Materials bind textures and scalar/vector inputs without ad hoc draw code. |
| FR-014 | Directional Lighting          | The forward renderer shall support at least one directional light.                              | C | Directional light affects visible shading in the sample scenes.            |
| FR-015 | Point Lighting                | The renderer shall support multiple point lights.                                               | H | Several point lights can illuminate a scene at once.                       |

| ID     | Title                          | Description                                                                                    | P | Acceptance Summary                                                           |
|--------|--------------------------------|------------------------------------------------------------------------------------------------|---|------------------------------------------------------------------------------|
| FR-016 | Forward Opaque Rendering       | The renderer shall implement a forward opaque pass for visible objects.                        | C | Main demo scenes render through the forward path only.                       |
| FR-017 | Directional Shadow Mapping     | The system shall generate and sample a directional shadow map.                                 | H | Validation scene shows stable shadows with tunable bias and filtering.       |
| FR-018 | Optional Basic CSM             | The project may include basic cascaded directional shadows as a controlled stretch goal.       | S | If enabled, cascades are toggleable and documented as stretch functionality. |
| FR-019 | Direct-Light PBR               | The material and shader pipeline shall support a direct-light metallic-roughness PBR workflow. | H | PBR test scene shows plausible roughness/metalness variation.                |
| FR-020 | Normal Mapping                 | The renderer shall support tangent-space normal maps.                                          | H | Normal maps visibly change local lighting detail.                            |
| FR-021 | Skybox Rendering               | The system shall render a skybox or cubemap background.                                        | M | Environment background renders correctly behind scene geometry.              |
| FR-022 | HDR Rendering                  | The renderer shall render the main scene into HDR-capable offscreen targets.                   | H | Bright values survive before tone mapping.                                   |
| FR-023 | Tone Mapping                   | The system shall apply configurable tone mapping to the HDR result.                            | H | Exposure and operator settings change final output predictably.              |
| FR-024 | Bloom                          | The renderer shall apply bloom as a post-process effect.                                       | H | Bright objects produce controllable bloom.                                   |
| FR-025 | Frustum Culling                | The renderer shall cull render items outside the camera frustum.                               | M | Optimization scene reduces submitted/visible draws without visible errors.   |
| FR-026 | Instanced Rendering            | The system shall support instanced draws for repeated meshes.                                  | M | Repeated objects render through an instanced path.                           |
| FR-027 | Debug UI and Diagnostics       | The system shall provide runtime controls, render statistics, and inspectors using ImGui.      | H | Debug panels expose renderer state, feature toggles, and metrics.            |
| FR-028 | Intermediate Target Inspection | The renderer shall expose shadow and post-process intermediate textures for debugging.         | M | Developers can inspect shadow map and HDR/bloom-related outputs.             |
| FR-029 | Logging and Error Reporting    | The system shall log startup, asset, shader, and runtime diagnostic events.                    | C | Errors are timestamped and visible in logs without silent failure.           |
| FR-030 | Configuration Presets          | The system shall support configuration presets for demo scenes and benchmark runs.             | M | Defined presets launch and reproduce target scenes/settings.                 |

## 5. Non-Functional Requirements

| ID      | Category    | Requirement                                                                                     | Rationale                                           | Acceptance Summary                                               |
|---------|-------------|-------------------------------------------------------------------------------------------------|-----------------------------------------------------|------------------------------------------------------------------|
| NFR-001 | Performance | The renderer shall maintain interactive frame rates on moderate demo scenes on target hardware. | Project viability depends on usable responsiveness. | Benchmark scenes and frame statistics are captured and reported. |

| ID      | Category                   | Requirement                                                                                                             | Rationale                                                     | Acceptance Summary                                                           |
|---------|----------------------------|-------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|------------------------------------------------------------------------------|
| NFR-002 | Modularity                 | Subsystems shall be separable into renderer core, resources, scene submission, lighting/shadows, post, and diagnostics. | The renderer must be maintainable and extensible.             | Codebase organization and interfaces reflect subsystem boundaries.           |
| NFR-003 | Maintainability            | Code shall follow documented conventions, reviewed merges, and ownership boundaries.                                    | Weekly delivery requires readable, reviewable code.           | Coding standard and review workflow are applied throughout.                  |
| NFR-004 | Extensibility              | The design shall allow later expansion to deferred rendering or additional backends without forcing it now.             | Future growth is a strategic goal.                            | No core interfaces hard-code application-specific assumptions unnecessarily. |
| NFR-005 | Reliability                | Startup, resize, scene switching, and shutdown shall avoid crashes under ordinary use.                                  | Final demo stability is critical.                             | Acceptance runs complete without high-severity failures.                     |
| NFR-006 | Debuggability              | The renderer shall surface meaningful logs, statistics, and debug visualizations.                                       | Graphics defects are hard to diagnose without built-in tools. | Debug panels and logs make failures observable.                              |
| NFR-007 | Reproducibility            | The build and run process shall be documented and repeatable on Windows.                                                | Team productivity depends on reproducibility.                 | A clean setup succeeds with documented steps.                                |
| NFR-008 | Documentation Quality      | The project shall provide up-to-date setup, architecture, API, and testing documentation.                               | The deliverable must support handoff and evaluation.          | Final documentation set is complete and versioned.                           |
| NFR-009 | Testability                | Core logic and scene configurations shall be amenable to scripted checks and repeatable validation scenes.              | Rendering projects still require disciplined testing.         | Checklists, scene presets, and smoke/regression procedures exist.            |
| NFR-010 | Usability of Internal APIs | Renderer-facing APIs shall be understandable to future engine programmers.                                              | Internal adoption depends on API clarity.                     | Submission and material/resource APIs are documented with examples.          |

## 6. System Architecture Requirements

### 6.1 Architectural Style

The renderer shall follow a modular subsystem architecture with explicit per-frame data flow. The design shall avoid burying rendering policy inside the application layer and shall avoid a monolithic source file structure.

### 6.2 Subsystems

Core subsystems are: Platform Layer; Renderer Core; Shader System; Buffer and Mesh System; Texture and Resource Management; Material System; Scene Submission Layer; Lighting and Shadows; Post-Processing; Debug UI and Diagnostics; Build/Test/Packaging Support.

### 6.3 Per-Frame Data Flow

A typical frame shall proceed as follows: begin frame -> consume configuration toggles -> receive camera/lights/render items -> perform visibility preparation -> render shadow pass -> render forward opaque scene -> render skybox -> execute HDR/tone-map/bloom -> render debug UI -> present.

## 6.4 Resource Ownership

Long-lived GPU resources shall be managed through explicit handles or owners rather than hidden global state. Shared assets such as textures, meshes, and shaders shall be cached. Per-frame or pass-local resources shall be clearly separated from persistent resources.

## 6.5 Error Handling Strategy

The renderer shall log recoverable failures, use assertions for programmer errors in debug builds, and fail fast when initialization prerequisites are not met. Missing assets should be diagnosable and use fallback assets only where the fallback meaning is explicit.

## 6.6 Codebase Organization

Source layout shall align with subsystem boundaries. Public headers, implementation files, assets, tools, test scenes, and documentation shall be separated to support handoff and future growth.

| Subsystem         | Key Responsibilities                                                          | Key Interfaces                          |
|-------------------|-------------------------------------------------------------------------------|-----------------------------------------|
| Renderer Core     | Initialization, frame orchestration, resize handling, present flow            | Renderer, RenderConfig, FrameContext    |
| Shader System     | Source loading, compile/link, shader variants kept minimal, program cache     | ShaderProgram, ShaderManager            |
| Geometry/Buffers  | VAO/VBO/IBO creation, primitive meshes, imported mesh buffers                 | Mesh, MeshBuffer, VertexLayout          |
| Texture/Resources | Texture loading, caches, sampler policy, framebuffer textures                 | Texture2D, ResourceCache, Framebuffer   |
| Materials         | Shader/material binding, material instances, scalar/vector/texture parameters | Material, MaterialInstance              |
| Scene Submission  | Camera, render-item list, transforms, light data input                        | FrameSubmission, RenderItem, CameraData |
| Lighting/Shadows  | Directional/point lights, shadow map pass, optional basic CSM stretch         | LightData, ShadowSettings               |
| Post-Processing   | HDR output, tone mapping, bloom and final composite                           | PostProcessPass, HDRSettings            |
| Diagnostics/UI    | ImGui panels, statistics, inspectors, debug toggles                           | DebugUI, RenderStats, Inspectors        |

## 7. Technology Stack and Third-Party Libraries

| Library / Tool | Subsystem                       | Why Selected                                                                         | Notes                                    |
|----------------|---------------------------------|--------------------------------------------------------------------------------------|------------------------------------------|
| GLFW           | Platform/<br>window/<br>context | Focused and lightweight for Windows desktop windowing and OpenGL context creation.   | Primary windowing library.               |
| GLAD           | OpenGL<br>loading               | Simple OpenGL function loading after context creation.                               | Generated for required API level.        |
| GLM            | Math                            | Standard vector/matrix math for cameras, transforms, frusta, and shading parameters. | Used throughout rendering math.          |
| Assimp         | Mesh import                     | Practical import path for static model formats.                                      | Used for imported static assets only.    |
| stb_image      | Texture<br>loading              | Lightweight image decoding for textures and cubemap faces.                           | Used with documented color-space policy. |

| Library / Tool | Subsystem                  | Why Selected                                                     | Notes                                           |
|----------------|----------------------------|------------------------------------------------------------------|-------------------------------------------------|
| Dear ImGui     | Debug UI/tools             | Rapid in-engine debug tooling, inspectors, and toggles.          | Used for runtime developer interface.           |
| spdlog         | Logging                    | Structured logging with severity levels and readable formatting. | Feeds startup and runtime diagnostics.          |
| nlohmann/json  | Configuration and metadata | Simple JSON-driven config, scene presets, and material metadata. | Avoids heavier serialization stack.             |
| CMake          | Build system               | Repeatable, IDE-friendly builds and dependency integration.      | Canonical build definition.                     |
| RenderDoc      | GPU debugging              | Critical for inspecting passes, state, and resources.            | Development/debug tool, not runtime dependency. |

## 8. Sprint Plan

Each sprint is one week.

### Sprint 1 - Project Setup, Platform Layer, and OpenGL Bootstrap

Objective: Establish the engineering baseline, reproducible build, Windows-targeted platform layer, and a working OpenGL application shell.

#### Feature Scope

- Repository structure, branching policy, and coding conventions
- CMake build with third-party dependency integration
- GLFW window creation and event loop
- GLAD loading and OpenGL debug context
- Logging, configuration, startup diagnostics, and clean shutdown

#### Detailed Engineering Tasks

1. Create repository layout for src/include/assets/docs/tests/tools and define naming, style, and review rules.
2. Integrate GLFW, GLAD, GLM, Assimp, stb\_image, ImGui, spdlog, and nlohmann/json through CMake with pinned versions or documented acquisition.
3. Implement application bootstrap, Win32-compatible window creation through GLFW, input polling, swap chain loop, and resize event plumbing.
4. Enable OpenGL debug output in debug builds and log GPU vendor, renderer, version, extensions summary, and active renderer configuration.
5. Create startup and shutdown lifecycle stubs for Application and Renderer objects, including deterministic destruction order.
6. Add configuration file loading for initial window size, VSync, log level, asset root, and debug toggles.

#### Expected Deliverables

- Project builds on target Windows machines
- OpenGL window opens, clears, resizes, and shuts down cleanly
- Startup log reports runtime environment and configuration

#### Definition of Done

- All required libraries integrate and compile under the documented build path
- Window creation, event polling, buffer swap, and clean shutdown succeed on at least two team machines
- Build and run instructions are committed and tested by non-authors



## **Sprint 2 - Minimal Renderer Core, Shaders, and First Geometry**

Objective: Replace bootstrap-only code with a reusable minimal renderer capable of drawing indexed geometry through managed shader, buffer, vertex layout and primitive abstractions.

### **Feature Scope**

- Renderer lifecycle API
- Shader loading/compilation/linking
- Vertex buffers, index buffers, VAOs, and vertex layout description
- Triangle, quad rendering
- Basic renderer-side error handling and diagnostics

### **Detailed Engineering Tasks**

1. Implement a ShaderProgram abstraction with source loading, stage compilation, link reporting, lifetime ownership, and reuse by identifier.
2. Implement Buffer and VertexArray wrappers for GPU resource lifetime management, plus a minimal MeshBuffer abstraction for vertex/index upload.
3. Define a minimal vertex layout system supporting at least position-only and position+color test geometry.
4. Create built-in triangle, quad, and cube primitive generation for testing of non-indexed and indexed rendering paths, extension to procedural 2D primitives such as circle and ellipse if sprint capacity allows.
5. Add a minimal render command path for begin frame, clear, bind pipeline state (shader program, blend state, depth test state, cull mode, ...), bind mesh, draw (indexed), and end frame.
6. Introduce a fallback shader strategy so shader failures are visible, diagnosable, and non-silent during development.
7. Add basic debug-time validation and logging for shader failures, invalid resource creation.
8. Separate application-layer logic from renderer implementation, and define stable public headers for renderer-facing use.

### **Expected Deliverables**

- Primitives(triangle, quad, cube, ...) render through the Renderer API
- Shader errors are human-readable and logged with stage and filename
- Renderer-side geometry submission is possible without direct OpenGL calls from application code

### **Definition of Done**

- Renderer initializes, renders built-in primitives, and shuts down without resource leaks
- Shader compile and link failures identify stage and source file
- Indexed rendering path is verified with at least one non-trivial primitive
- Primitive types can be rendered through the same renderer-facing draw path
- Sample application code depends only on public renderer headers, not internal implementation details

## Sprint 3 - Camera, Transforms, Scene Submission, and Debug Controls

Objective: Turn the renderer from “it can draw primitives” into “it can render an actual 3D scene”. This sprint should introduce a camera, per-object transforms and a scene submission flow so the application stops issuing low-level draw intent directly and instead describes what should be rendered each frame.

### Feature Scope

- Perspective camera
- Transform system
- Scene/render submission structures
- Per-frame GPU constants
- Basic debug UI and runtime toggles
- Simple 3D scene

### Detailed Engineering Tasks

1. Introduce a camera that is actually usable in a 3D scene. Implement a camera type that owns or can compute: position, orientation, forward/right/up vectors, field of view, aspect ratio, near and far clip planes, view matrix, projection matrix. It should support free-fly, first-person camera, third-person camera.
  - a) Separate camera state from input polling.
  - b) Recompute projection matrix when framebuffer size changes
  - c) Clamp pitch if using mouse-look to avoid flipping
  - d) Keep movement frame-rate independent using delta time
2. Add Transform data structure with translation, rotation, scaling, and model-matrix generation. Euler angles or quaternions can be used. Non-uniform scale should be supported.
3. Create submission-side data types such as: CameraData, RenderItem, FrameSubmission. RenderItem should describe:
  - which mesh or primitive to render
  - which shader/material or pipeline state to use
  - transform/model matrix
  - draw mode and topology
  - visibility flags

FrameSubmission should describe:

- active camera
  - viewport/framebuffer size
  - clear color and clear policy
  - collection of render items for the frame
  - optional debug draw requests (wireframe, ...).
4. Upload per-frame view/projection/camera data through uniform blocks (view matrix, projection matrix, camera position, combined view-projection, timing data if needed, per object model and normal matrix).
  5. Refine the renderer API, a frame should be like this:
    - I. begin frame
    - II. accept FrameSubmission
    - III. upload frame constants
    - IV. iterate render items
    - V. bind required state
    - VI. issue draw calls
    - VII. render debug UI
    - VIII. end frame / present
  6. Integrate lightweight ImGui debug overlay for: frame time / FPS, camera position and orientation, current framebuffer size, toggles for wireframe, count of submitted render items

7. Create simple test scene composed of built-in primitives positioned through the submission interface. For example:
  - a) several cubes and quads placed at different positions
  - b) different scales and rotations
  - c) one or two rows of primitives to visually confirm transforms
  - d) a ground plane
  - e) a camera starting position that immediately shows depth and perspective

#### **Expected Deliverables**

- Navigable 3D test scene
- Render-item submission API used by the sample application
- Basic debug overlay showing frame stats and toggles

#### **Definition of Done**

- The sample app builds a FrameSubmission every frame instead of issuing ad hoc direct draw intent
- Multiple objects render with distinct transforms through the same renderer-facing scene path
- Camera movement and look controls are stable and intuitive
- Projection updates correctly on resize without stretching or broken clipping
- Renderer-owned frame/global data is not scattered across arbitrary uniform calls
- Debug overlay works without corrupting rendering or input flow
- The application depends only on public renderer interfaces, not renderer internals

## Sprint 4 - Mesh Import, Textures, and Material System

Objective: This sprint should establish the first real asset workflow: mesh import, texture loading, basic materials, caching, and rendering of multi-submesh models.

### Feature Scope

- Mesh import with Assimp
- 2D texture loading with stb\_image
- Mesh and texture resource caches
- Material and material instance abstractions
- Imported model rendering through the existing scene submission path
- Fallback behavior for missing assets

### Detailed Engineering Tasks

1. Integrate Assimp for the following model formats: 1. OBJ, 2. glTF (<https://github.com/KhronosGroup/glTF-Sample-Models>). The mesh import path should extract: positions, normals, UVs, tangents/bitangents, indices, submesh boundaries.
2. Integrate stb\_image for texture loading and convert decoded image data into GPU textures. Support:
  - a) base color / albedo textures
  - b) fallback single-color textures when files are missing
  - c) mip generation
  - d) Flip-on-load policy must be explicit and consistentAlso define a basic color space policy: 1. color/albedo textures should usually be treated as sRGB, 2. non-color data like normal/roughness/metallic should remain linear.
3. Implement Texture2D and Mesh resource caches keyed by logical asset path to avoid duplication of assets.
4. Define Material and MaterialInstance structures containing shader selection, scalar/vector parameters, bound textures, whether the material expects textured or untextured rendering. Imported meshes should render because their material describes what to bind, not because the application manually wires each texture every frame.
5. Extend scene submission so imported assets fit the same path as primitives.
6. Document asset folder conventions, texture expectations, and fallback behavior. Use something like:
  - assets/models/...
  - assets/textures/...
  - assets/materials/... if applicable
  - sample models with associated textures
7. Build a model-loading sample scene.

### Expected Deliverables

- Imported textured meshes render correctly
- Resource caches prevent duplicate loads
- Material definitions replace hard-coded per-draw texture setup
- A sample scene containing multiple imported model with textures, navigable with camera.

### Definition of Done

- External model loads and renders correctly with textures
- Models with multiple submeshes/material slots render through the renderer's normal scene path
- Duplicate requests for the same mesh or texture reuse cached resources instead of recreating them
- Missing textures or invalid asset references fail with log information.
- Built-in primitive rendering still works and has not been replaced by a separate "asset-only" path.
- Material binding is no longer hardcoded
- The project now has a clear asset-loading workflow

## Sprint 5 - Forward Lighting and Production Frame Structure

Objective: Build a usable lit forward renderer with clear per-frame organization, directional and point lights, and better runtime diagnostics.

### Feature Scope

This sprint focuses on the first practical forward lighting path for the renderer. The scope includes:

- forward opaque rendering path suitable for real scene rendering,
- support for directional and point lights,
- clean flow of per-frame and per-object data into the GPU,
- runtime statistics and diagnostics for visibility into renderer behavior,
- and basic debug UI controls for working with lighting and renderer settings interactively.

### Detailed Engineering Tasks

1. Forward opaque rendering pass: Build the forward-rendering path for opaque objects with a predictable, documented render queue to keep material/shader binding stable and efficient. Establish a clear pattern for gathering, ordering, and submitting visible render items.
2. Light submission structures: Add data structures and submission flow for one directional light and multiple point lights per frame. Define a clean boundary between CPU-side scene/light setup and GPU-side shader consumption to support future light types (spot lights, area lights, ...).
3. Direct-light shading baseline: Implement an initial direct-lighting path for the supported light types in this sprint. Use a simple, reliable shading model for now, but organize the renderer around clear GPU inputs for frame data, object transforms, material parameters, and light parameters. Material and light structures, shader bindings, and naming should follow a layout that can later be extended for full PBR features such as metallic, roughness, normal mapping, emissive, and improved BRDF-based lighting, without requiring the renderer submission path or shader interface model to be redesigned.
4. Per-frame execution structure: Keep the renderer organization in clear frame lifecycle (begin frame → scene pass → overlays/debug → present) to make execution easier to reason about, debug, and extend.
5. Runtime statistics: Track key metrics internally: draw call count, visible object/triangle counts, loaded resource totals, and frame timing.
6. Lighting and renderer debug controls: Extend the debug UI with runtime light controls (placement, color, intensity) and renderer toggles for inspecting behavior during scene testing, without requiring code changes per adjustment.

### Expected Deliverables

- Forward opaque renderer with directional and point lights
- Stable frame structure with visible runtime statistics
- Lighting can be manipulated interactively through the debug UI

### Definition of Done

- Opaque lit scenes render correctly with at least one directional light and multiple point lights active simultaneously
- Lighting results are stable and produce visually testable output; no flickering, incorrect attenuation, or missing contributions
- Render queue sort order is implemented consistently and documented so the ordering logic is clear to any engineer picking up the code
- Frame statistics are accurate within reasonable tolerance and reflect actual scene complexity: draw call count, visible object count, visible triangle count, loaded resource totals and frame timing are all tracked and displayed
- Frame lifecycle phases are clearly separated in code and easy to follow; a new engineer should be able to identify where to add a new pass without ambiguity
- Debug UI light controls are functional and cover at minimum placement, color, and intensity for all supported light types

## Sprint 6 - Directional Shadow Mapping

Objective: Add shadow-map generation and sampling to the forward renderer, with a realistic path toward optional basic cascades.

### Feature Scope

- Shadow map resources
- Directional light shadow pass
- Shadow sampling in forward pass
- Bias/filter tuning
- Shadow debug views

### Detailed Engineering Tasks

1. Depth-only framebuffer and shadow texture creation: Set up the framebuffer and texture resources needed for a directional light shadow map. Resolution should be a configurable parameter, not hard-coded, and the resource lifecycle (creation, binding, cleanup) should be explicit and cleanly separated from the main scene framebuffer.
2. Light-space camera and shadow-caster render path: Generate a view/projection matrix from the directional light's perspective and render shadow-casting geometry into the depth-only framebuffer as a distinct pass before main scene shading. The pass should reuse existing mesh submission infrastructure where possible but remain clearly separated in the frame lifecycle, with no inherited shading state or material bindings from the main pass.
3. Shadow map sampling in the forward pass: Sample the shadow map during forward shading to attenuate direct lighting for shadowed fragments. Bias should be configurable to allow tuning against self-shadowing artifacts, and PCF should be implemented to soften shadow edges. Filter kernel size and sample count should be exposed as tunable parameters rather than fixed in the shader.
4. Runtime debug controls and shadow map inspection: Extend the debug UI to expose shadow parameters: resolution, depth bias, PCF filter settings, and light-space frustum bounds. Add at least one debug view that renders the shadow map directly to screen so artifacts can be diagnosed without an external tool like RenderDoc.
5. Shadow-caster flags on render items: Add a per-render-item flag controlling shadow pass participation. Objects without it should be explicitly excluded from shadow-caster submission, both for correctness and to avoid unnecessary draw calls. The flag should be settable at the object or material level and visible in debug output.
6. Cascaded shadow map extension path (stretch goal): Design the shadow system with CSM in mind. Document where cascade splits would be introduced, how light-space camera generation would change, and how the forward sampler would select the correct cascade.

### Expected Deliverables

- Directional shadow mapping integrated into the forward renderer
- Shadow map debug inspection tools
- Documented stretch plan for basic cascaded shadows

### Definition of Done

- A validation scene renders stable directional shadows across target camera positions, with no flickering, missing shadow contributions, or gross self-shadowing artifacts under normal bias settings
- Shadow map resolution, depth bias, and PCF filter settings are all adjustable at runtime through the debug UI with no recompile required, and the purpose and recommended range of each parameter is documented
- The shadow pass and main shading pass are individually diagnosable: the shadow map can be inspected directly on screen, shadow-caster submission can be traced, and a new engineer should be able to isolate a shadow artifact to either the depth pass or the sampling/filtering stage without needing to instrument the code from scratch
- Shadow-caster flags are functional and exclude non-casting objects from the shadow pass, confirmed through runtime statistics or debug output rather than assumed
- Light-space frustum fit is visible and inspectable in the debug UI, so incorrect frustum coverage can be identified and corrected without guesswork

## Sprint 7 - PBR and Normal Mapping

**Objective:** This sprint upgrades the renderer to use Physically Based Rendering (PBR): a shading model grounded in real-world physics that describes how light actually interacts with different materials. It also introduces normal mapping, a technique that makes flat surfaces appear to have fine surface detail (scratches, bumps, grooves) without adding any extra geometry.

### Background

PBR replaces previous Blinn-Phong model with a model built on measurable physical properties. The idea is: if inputs are physically correct, the output will look correct under any lighting condition automatically. This is why every modern game uses PBR.

#### What is a BRDF?

BRDF stands for Bidirectional Reflectance Distribution Function. It is a function that takes a light direction and a view direction and returns how much light is reflected toward the viewer from that surface point. Every shading model is a BRDF. Blinn-Phong is a BRDF. The PBR model is a more physically accurate BRDF.

The PBR BRDF we will use in this sprint is called **Cook-Torrance microfacet BRDF**. It models the surface as covered in millions of microscopic mirror-like facets. The orientation distribution of those facets is controlled by the roughness value. When roughness is low, most facets are aligned and reflect light in the same direction, producing a sharp highlight. When roughness is high, the facets point in many directions, scattering light broadly.

The full BRDF has three components:

**1) D – Normal Distribution Function (NDF):** How many microfacets are oriented to reflect light toward the camera. **GGX is the standard for this.**

**2) G – Geometry Function:** Accounts for microfacets shadowing each other (self-shadowing at grazing angles). This prevents surfaces from appearing unrealistically bright when viewed edge-on. Use Smith Geometry function for this part.

**3) F – Fresnel Term:** Describes how reflectivity changes with viewing angle. Every surface becomes more reflective as you look at it from a shallow angle; wet roads reflect light when viewed along their length but not when looked straight down. **Schlick's approximation** is the standard implementation.

Then, D, G, and F terms are divided by  $4 * NdotV * NdotL$  to keep the BRDF energy-consistent:  
 $specularBRDF = (D * G * F) / (4 * NdotV * NdotL)$ .

#### What are two main PBR inputs - Metallic and Roughness?

PBR in the metallic-roughness workflow describes every surface using two fundamental properties:

**1) Metallic (0.0 to 1.0)** - This describes whether a surface is a metal or a non-metal (also called a dielectric).

0.0 = completely non-metal (stone, wood, skin, plastic, fabric)

1.0 = completely metal (iron, gold, copper, aluminium)

The physical distinction matters because metals and non-metals reflect light differently at a fundamental level. Non-metals reflect light diffusely (scatter it in all directions). Metals reflect light specularly (in a mirrored direction) and also tint the reflection with their surface color. Pure gold reflects gold-tinted specular highlights. Pure iron reflects white-grey specular highlights. In practice, most real-world surfaces are either fully metal or fully non-metal. Values in between are sometimes used to blend for artistic reasons, but physically they represent a surface that is partially corroded or coated.

**2) Roughness (0.0 to 1.0)** - This describes the microscopic texture of a surface at a scale too small to model with geometry.

0.0 = perfectly smooth mirror (polished chrome ball, still lake)

1.0 = completely rough (chalk, unfinished concrete, raw wood)

At the microscopic level, no surface is perfectly flat. Even glass has tiny imperfections. These micro-imperfections scatter incoming light slightly. The rougher a surface is, the more the light scatters, which spreads out and blurs specular highlights. A low-roughness surface has a small, sharp, bright specular highlight. A high-roughness surface has a large, dim, spread-out specular highlight.

## What is Normal Map?

A surface normal is a vector that points directly away from a surface. The renderer uses this vector to calculate how light hits the surface. By default, every point on a flat polygon shares the same normal, perpendicular to the polygon's surface. This means the entire polygon reacts identically to lighting, which makes it look flat.

A normal map is a texture where each pixel stores a slightly different normal direction encoded as a color. When the shader reads the normal map instead of using the default surface normal, each pixel on the surface appears to have its own orientation. The result is that the surface appears to have fine detail (grooves, bumps, scratches) that reacts correctly to light from any direction. The geometry of the mesh has not changed. No extra polygons have been added. The illusion is entirely created by changing the direction the lighting calculation thinks the surface is facing.

## What is Tangent Space?

Normal maps store normals relative to the surface itself, not relative to the world. This is called **tangent space**.

If we have a brick wall tilted at 45 degrees, we want the same normal map to work correctly regardless of the wall's orientation. If normals were stored in world space, the same normal map applied to a wall facing north would be wrong when applied to a wall facing east.

To solve this, each vertex on the mesh carries three vectors that define a local coordinate frame:

- 1) **Normal (N)**: Points away from the surface
- 2) **Tangent (T)**: Points along the surface in the U direction of the UV map
- 3) **Bitangent (B)**: Points along the surface in the V direction of the UV map (also known as Binormal)

Together, these form the TBN matrix. In the shader, we must transform the normal map sample from tangent space into world space by multiplying it by this TBN matrix. The result is a world-space normal for lighting calculations.

## What is Albedo Map (Base Color)?

Albedo (or base color) is the color of the surface without any lighting applied. It is the raw pigment of the material, what color it would be in neutral lighting.

## What is Ambient Occlusion Map?

It is a greyscale texture that stores how much of the sky (ambient light) each part of a surface can see. Crevices and corners that are surrounded by geometry receive less ambient light and are stored as darker values. Open exposed areas receive more and are stored as brighter values. AO does not affect direct lighting and it is only applied to the ambient (indirect) light term.

## What is Emissive Map?

Some surfaces emit light of their own: screens, LEDs, neon signs. An emissive map stores the color and intensity of this self-emission. In the shader, the emissive value is added directly to the final output without being multiplied by any lighting term, as emitted light does not depend on external illumination.

## Feature scope

- Metallic-roughness PBR – replace the existing lighting model with a Cook-Torrance BRDF
- Normal mapping – per-pixel normal perturbation from a texture
- Tangent-space support – TBN matrix construction and transformation in shaders, with CPU-side tangent generation for meshes that lack them (assimp can do this)
- Expanded material schema – materials now store: albedo, normal, metallic, roughness, AO (optional), emissive (optional)
- PBR validation scene – a test scene demonstrating the full parameter range to verify correctness

## Detailed Engineering Tasks

1. Implement the Cook-Torrance BRDF in GLSL replacing the existing lighting shader with a physically based BRDF that handles both diffuse and specular contributions using the metallic-roughness parameterisation.
2. Expand the material schema to hold all the PBR texture slots and scalar parameters. Each texture slot must be bound to a consistent texture unit (for example: albedo = unit 0, normal = unit 1, metallic = unit 2, roughness = unit 3, AO =



unit 4, emissive = unit 5). Document these bindings. Use a uniform flag (like *bool uHasNormalMap*) to allow the shader to fall back when a texture is not provided. What the new material structure should contain:

| Field          | Type      | Description                     |
|----------------|-----------|---------------------------------|
| albedoMap      | Texture2D | Base colour (no baked lighting) |
| albedoColor    | vec3      | Fallback colour if no texture   |
| normalMap      | Texture2D | Tangent-space normal map        |
| metallicMap    | Texture2D | Metallic values (greyscale)     |
| metallicValue  | float     | Fallback if no texture          |
| roughnessMap   | Texture2D | Roughness values (greyscale)    |
| roughnessValue | float     | Fallback if no texture          |
| aoMap          | Texture2D | Ambient occlusion (optional)    |
| emissiveMap    | Texture2D | Emissive colour (optional)      |
| emissiveColor  | vec3      | Emissive tint/intensity         |

3. Tangent Space Support – We must make sure that every mesh that will use normal mapping has valid tangent data, either imported from the asset file or computed at load time.

a) Check for tangents on import. When loading a mesh (using Assimp), check if the mesh includes tangent and bitangent data. Assimp can generate them automatically with relevant flag.

b) Store tangents in the vertex buffer.

c) Build the TBN matrix in the vertex shader.

```
vec3 world_normal = normalize(u_NormalMatrix * a_Normal);
vec3 world_tangent = normalize(u_NormalMatrix * a_Tangent.xyz);
world_tangent = normalize(world_tangent - dot(world_tangent, world_normal) * world_normal); // Gram-Schmidt
const vec3 world_bitangent = normalize(cross(world_normal, world_tangent) * a_Tangent.w);

v_TBN = mat3(world_tangent, world_bitangent, world_normal);
```

d) Sample the normal map in the fragment shader for lightning computation.

```
vec3 ResolveWorldNormal()
{
    if (u_HasNormalTexture == 0)
    {
        return normalize(v_WorldNormal);
    }

    vec3 tangent_space_normal = texture(u_NormalTexture, v_TexCoord0).xyz * 2.0 - 1.0;
    tangent_space_normal.xy *= u_NormalScale;
    tangent_space_normal.z = sqrt(max(1.0 - dot(tangent_space_normal.xy, tangent_space_normal.xy), 0.0));
    return normalize(v_TBN * tangent_space_normal);
}
```

4. Update shaders and material binding paths for normal maps and PBR parameters.
5. Build a test scene specifically designed to verify that PBR implementation is correct across its full parameter range. Arrange a 5×5 grid of spheres (25 total): X axis varies roughness from 0.0 (left) to 1.0 (right), Y axis varies metallic from 0.0 (bottom) to 1.0 (top). All spheres share the same albedo (grey: 0.5, 0.5, 0.5). Use a single directional light

from a fixed angle.

Also, add a gold-coloured sphere (albedo 1.0, 0.766, 0.336, metallic 1.0, roughness 0.1), it should have specular highlight visibly gold-tinted, not white.

Also, add a red sphere (metallic 0.0, roughness 0.5) , the specular highlight should be white, not red (non-metals always have white highlight).

### **Expected Deliverables**

- Direct-light PBR shading in the forward renderer
- Normal mapping on imported assets
- PBR material samples and authoring notes

### **Definition of Done**

- The PBR validation scene (5×5 sphere grid) shows smooth variation in both metallic and roughness dimensions with no flat shading, uniform brightness, or missing highlights
- A gold sphere and a red sphere display correct specular tint behaviour (gold tinted, red untinted)
- Normal maps visibly affect lighting detail on test assets.
- All six material texture slots are documented, bound to consistent units, and fall back gracefully when not provided

## **Sprint 8 - Skybox, HDR, Tone Mapping, and Bloom**

Objective: Deliver a modern final-image pipeline on top of the forward PBR renderer.

### **Feature Scope**

- Skybox/environment background
- HDR scene rendering
- Tone mapping
- Bloom
- Intermediate target inspection

### **Detailed Engineering Tasks**

1. Implement cubemap loading and skybox rendering that integrates correctly with depth handling and camera rotation.
2. Move main scene rendering to HDR-capable offscreen targets.
3. Implement at least one tone-mapping operator with exposure control and debugging UI.
4. Implement bloom using bright-pass extraction, blur stages, and compositing into the final image.
5. Add debug inspection for HDR color, bloom threshold, and final LDR output.
6. Keep the post-process framework simple, explicit, and maintainable rather than overgeneralized.

### **Expected Deliverables**

- Skybox/environment background
- HDR rendering path with tone mapping
- Bloom integrated into the final image pipeline

### **Definition of Done**

- Scene renders through HDR targets and displays through configurable tone mapping
- Bloom is clearly visible, controllable, and can be disabled for comparison
- Intermediate targets can be inspected for debugging

## **Sprint 9 - Optimization, Robustness, and Developer Tooling**

Objective: Improve scalability, stability, and development usability without widening core scope.

### **Feature Scope**

- Frustum culling
- Instanced rendering
- Queue/state optimization
- Profiling/statistics upgrades
- Resource cleanup hardening

### **Detailed Engineering Tasks**

1. Implement CPU-side frustum culling for submitted render items using camera frustum tests and bounding volumes.
2. Implement instanced rendering for repeated meshes with per-instance transform data.
3. Improve render queue sorting to reduce redundant material/shader/state changes.
4. Expand debug overlays with pass timings, culling stats, resource counts, and capture-friendly scene presets.
5. Audit resource lifetime and shutdown paths to eliminate leaks and double-free risks.
6. As a stretch only if stable, prototype basic cascaded directional shadows behind an isolated feature toggle.

### **Expected Deliverables**

- Frustum culling and instanced rendering
- Improved diagnostics and benchmark presets
- More robust resource lifetime and shutdown behavior

### **Definition of Done**

- Optimization scenes show measurable reduction in draw count or frame cost compared with the pre-optimization baseline
- No critical regressions appear in lighting, shadows, PBR, or post-processing after optimization changes
- Acceptance test checklist passes on the intended demo scenes

## **Sprint 10 - Stabilization, Documentation, Testing, and Final Delivery**

Objective: Freeze scope, harden quality, complete documentation, and package the final renderer for evaluation.

### **Feature Scope**

- Bug fixing and refactoring
- Acceptance validation
- Benchmarks
- Final docs
- Release packaging and presentation

### **Detailed Engineering Tasks**

1. Resolve highest-priority defects and refactor unstable interfaces without changing agreed scope.
2. Run full acceptance checklist across all required demo scenes and renderer toggles.
3. Capture benchmark results, screenshots, and if desired RenderDoc-based evidence for selected scenes.
4. Complete developer setup guide, architecture overview, subsystem docs, sprint evidence, and final limitations list.
5. Package assets, configuration presets, executable/build instructions, and presentation material.
6. Conduct final release rehearsal on a clean machine or clean workspace using only documented steps.

### **Expected Deliverables**

- Release candidate and final package
- Complete documentation set
- Benchmarks, demo scenes, and acceptance evidence

### **Definition of Done**

- A clean build and run succeeds using only the final documented instructions
- All critical acceptance criteria pass on the designated final demo scenes
- Final deliverables are archived, versioned, and presentation-ready

## Appendix A. Directory Structure

```
/renderer-forward
/assets
  /models
  /textures
  /cubemaps
  /materials
  /scenes
  /shaders
    /common
    /forward
    /shadow
    /postfx
    /debug
/config
/docs
/external
/src
  /app
  /platform
  /core
  /graphics
    /buffers
    /textures
    /shaders
    /materials
    /lighting
    /shadows
    /postfx
    /debug
  /scene
  /assets
  /ui
  /utils
/tests
/tools
CmakeLists.txt
```

## Appendix B. Final Demo Scenes

1. PBR material gallery with varying roughness and metalness
2. Shadow validation scene with strong sunlight direction and varied geometry silhouettes
3. Interior or courtyard scene with imported props and multiple point lights
4. HDR and bloom showcase scene with emissive accents and exposure controls
5. Optimization scene with many repeated objects to exercise instancing and culling

## Appendix C. Sprint Summary Table

| Sprint | Theme                                                     | Primary Outcome                                                                                                                                  |
|--------|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| 1      | Project Setup, Platform Layer, and OpenGL Bootstrap       | Establish the engineering baseline, reproducible build, Windows-targeted platform layer, and a working OpenGL application shell.                 |
| 2      | Minimal Renderer Core, Shaders, and First Geometry        | Replace bootstrap-only code with a reusable minimal renderer capable of drawing indexed geometry through managed shader and buffer abstractions. |
| 3      | Camera, Transforms, Scene Submission, and Debug Controls  | Introduce a real 3D scene flow with camera control, object transforms, and a renderer-facing scene submission API.                               |
| 4      | Mesh Import, Textures, and Material System                | Move from primitives to imported textured assets rendered through a reusable material pipeline.                                                  |
| 5      | Forward Lighting and Production Frame Structure           | Build a usable lit forward renderer with clear per-frame organization, directional and point lights, and better runtime diagnostics.             |
| 6      | Directional Shadow Mapping                                | Add shadow-map generation and sampling to the forward renderer, with a realistic path toward optional basic cascades.                            |
| 7      | PBR and Normal Mapping                                    | Upgrade the renderer from baseline direct lighting to a modern direct-light PBR workflow with normal-mapped materials.                           |
| 8      | Skybox, HDR, Tone Mapping, and Bloom                      | Deliver a modern final-image pipeline on top of the forward PBR renderer.                                                                        |
| 9      | Optimization, Robustness, and Developer Tooling           | Improve scalability, stability, and development usability without widening core scope.                                                           |
| 10     | Stabilization, Documentation, Testing, and Final Delivery | Freeze scope, harden quality, complete documentation, and package the final renderer for evaluation.                                             |

## Appendix D. Tutorials/references

Knjige:

1. Real-Time Rendering: [http://mutantstargoat.com/~nuclear/tmp/realtime\\_rendering\\_4ed.pdf](http://mutantstargoat.com/~nuclear/tmp/realtime_rendering_4ed.pdf)
2. OpenGL Programming Guide: <https://www.cs.utexas.edu/~fussell/courses/cs354/handouts/Addison.Wesley.OpenGL.Programming.Guide.8th.Edition.Mar.2013.ISBN.0321773039.pdf>

Ostalo:

- Tutorials: <https://learnopengl.com/>
- Tutorials: <https://ogldev.org/index.html>
- Tutorials: <https://www.youtube.com/@VictorGordan/videos>
- Starting-out, setup (sprint 1): <https://www.youtube.com/watch?v=Z2MiPOepwPE>
- RenderDoc: <https://www.youtube.com/watch?v=wPl0bcfpny8>
- ImGui integration tutorial: <https://www.youtube.com/watch?v=nVaQuNXueFw>
- PBR: <https://www.youtube.com/watch?v=RRE-F57fbXw>
- PBR, Camera: <https://www.youtube.com/watch?v=snNfQzi1608>